

Documentação do Sistema de Otimização de Escalas de Funcionários

Aplicação web para generalizar e resolver problemas de escala com n períodos, demanda mínima por período e restrições de trabalho/folga.

Frontend: React · Backend: Java · API REST · Solver: Google OR-Tools GLOP · Método: Simplex

1. Visão geral do projeto

O projeto consiste em uma aplicação web para resolver problemas de escala de funcionários usando otimização matemática. O usuário informa uma quantidade variável de períodos de escala, a demanda mínima de funcionários para cada período e as regras de trabalho e folga. A partir desses dados, o backend gera padrões possíveis de escala, monta um modelo de programação linear e calcula uma solução ótima usando o solver GLOP do Google OR-Tools.

A aplicação continua usando o problema da LCL Correios e Malotes como exemplo base, mas deixa de estar limitada aos sete dias da semana. O mesmo sistema pode representar uma escala semanal simples, turnos de manhã/tarde/noite, plantões 12/36, escalas de pilotos ou qualquer outro cenário dividido em n períodos ordenados.

Como o objetivo acadêmico do projeto é demonstrar o funcionamento do algoritmo Simplex, o sistema utilizará o solver **GLOP**, voltado para problemas de programação linear contínua. Portanto, as variáveis de decisão serão modeladas como variáveis não negativas contínuas. Caso fosse necessário garantir quantidades inteiras em uma aplicação real, o modelo poderia ser adaptado futuramente para MIP ou CP-SAT.

2. Problema original usado como base

O problema base considera uma empresa que precisa contratar funcionários para atender uma demanda mínima diária. Os funcionários trabalham em regime integral e seguem uma regra de escala fixa: trabalham uma quantidade de dias consecutivos e depois folgam uma quantidade de dias consecutivos.

Período	Demanda mínima
Segunda	18
Terça	12
Quarta	15
Quinta	19
Sexta	14
Sábado	16
Domingo	11

No exemplo original, cada funcionário trabalha 5 dias consecutivos e folga 2 dias consecutivos. A escala é circular, isto é, domingo conecta com segunda. O objetivo é minimizar o total de funcionários contratados.

Na versão generalizada, esses sete dias são apenas um caso particular em que $n = 7$. Em outros casos, cada período pode representar um turno ou um plantão, por exemplo: Segunda 07h-19h, Segunda 19h-07h, Terça 07h-19h e assim por diante.

3. Objetivos da aplicação

Objetivo geral: criar uma aplicação web capaz de calcular automaticamente a quantidade mínima de funcionários necessária para atender uma demanda por período, respeitando regras de trabalho e folga e demonstrando o uso de programação linear resolvida por Simplex.

Objetivos específicos:

- Permitir que o usuário informe n períodos de escala.
- Permitir que o usuário informe a demanda mínima de cada período.
- Permitir configurar regras de trabalho e folga em quantidade de períodos.
- Gerar automaticamente padrões possíveis de escala.
- Resolver o problema usando Google OR-Tools GLOP no backend.
- Apresentar total mínimo de funcionários, padrões, cobertura por período e gráficos.

- Exibir o modelo matemático conceitual gerado a partir dos dados informados.

4. Escopo do sistema

Dentro do escopo: cadastro de cenários; cadastro de n períodos ordenados; cadastro da demanda mínima de cada período; configuração da regra de trabalho e folga; geração automática dos padrões; montagem do modelo de programação linear; resolução com OR-Tools GLOP; exibição dos resultados em tabela e gráficos.

Fora do escopo inicial: controle real de ponto; cadastro individual de funcionários; integração com folha de pagamento; férias, atestados e licenças; preferências individuais; resolução inteira obrigatória com MIP; múltiplos tipos de contrato no mesmo cenário.

5. Premissas e decisões técnicas

Decisão	Justificativa
React no frontend	Facilita criação de telas dinâmicas, tabelas editáveis, pré-visualização de padrões e gráficos.
Java no backend	Boa organização em camadas e boa integração com OR-Tools.
API REST	Comunicação simples entre frontend e backend por JSON.
OR-Tools GLOP	Permite resolver programação linear contínua usando Simplex.
PostgreSQL	Permite armazenar cenários, períodos, regras e resultados.
n períodos	Permite representar semanas comuns, turnos, plantões 12/36 e outros horizontes de escala.
Variáveis contínuas	Mantém o foco didático em programação linear e Simplex.

6. Regras de negócio

- **RN01:** um cenário representa um problema completo de escala e deve possuir nome, descrição opcional, períodos, demandas e regra de trabalho/folga.
- **RN02:** o usuário pode cadastrar n períodos de escala, desde que eles possuam ordem definida.
- **RN03:** cada período deve possuir nome, ordem e demanda mínima maior ou igual a zero.
- **RN04:** a regra principal define quantidade de períodos consecutivos trabalhados, quantidade de períodos consecutivos de folga e se a escala é circular.
- **RN05:** os padrões de escala devem ser gerados automaticamente com base nos períodos e na regra de trabalho/folga.
- **RN06:** quando a escala for circular, o último período se conecta ao primeiro.
- **RN07:** para cada período, a soma dos funcionários alocados em padrões que trabalham naquele período deve ser maior ou igual à demanda mínima.
- **RN08:** o objetivo do modelo é minimizar o total de funcionários necessários.
- **RN09:** a solução do GLOP pode conter valores fracionários; a interface deve deixar claro que é uma solução linear contínua.
- **RN10:** caso o solver não encontre solução viável, o sistema deve informar o usuário.

7. Modelo matemático conceitual

Conjuntos: P = conjunto de períodos de escala; K = conjunto de padrões de escala gerados.

Parâmetros: d_p = demanda mínima do período p ; $a_{pk} = 1$ se o padrão k trabalha no período p , e 0 caso contrário.

Variáveis de decisão: x_k = quantidade de funcionários alocados no padrão k . No caso de uma escala circular simples, normalmente são gerados n padrões e, portanto, n variáveis, uma para cada posição inicial de folga ou trabalho no ciclo.

x_1 = funcionários no padrão 1
 x_2 = funcionários no padrão 2
 x_3 = funcionários no padrão 3
 ...
 x_n = funcionários no padrão n

Função objetivo: minimizar o total de funcionários.

$$\text{Min } Z = x_1 + x_2 + x_3 + \dots + x_n$$

Restrições de cobertura: para cada período, os funcionários trabalhando devem atender ou superar a demanda mínima.

$$\sum a_{pk} * x_k \geq d_p, \text{ para todo período } p$$

Não negatividade:

$$x_k \geq 0, \text{ para todo padrão } k$$

8. Geração de padrões de escala

A aplicação não deve deixar os padrões fixos no código. Eles devem ser gerados automaticamente a partir dos n períodos e das regras informadas.

Para o exemplo com 7 períodos, 5 trabalhadores e 2 de folga, os padrões seriam:

Padrão	Seg	Ter	Qua	Qui	Sex	Sáb	Dom
Folga Seg-Ter	0	0	1	1	1	1	1
Folga Ter-Qua	1	0	0	1	1	1	1
Folga Qua-Qui	1	1	0	0	1	1	1
Folga Qui-Sex	1	1	1	0	0	1	1
Folga Sex-Sáb	1	1	1	1	0	0	1
Folga Sáb-Dom	1	1	1	1	1	0	0
Folga Dom-Seg	0	1	1	1	1	1	0

Legenda: 1 = trabalha no período; 0 = folga no período. O solver decide quantos funcionários devem seguir cada padrão.

Em uma escala 12/36, por exemplo, os períodos podem ser blocos de 12 horas. Assim, o mesmo algoritmo de geração passa a trabalhar sobre a lista ordenada de plantões em vez de trabalhar apenas sobre dias da semana.

9. Requisitos funcionais

Código	Requisito
RF01	O sistema deve permitir criar um cenário de escala.
RF02	O sistema deve permitir cadastrar, editar, remover e ordenar n períodos de escala.
RF03	O sistema deve permitir informar a demanda mínima de cada período.
RF04	O sistema deve permitir configurar a quantidade de períodos trabalhados consecutivos.
RF05	O sistema deve permitir configurar a quantidade de períodos de folga consecutivos.
RF06	O sistema deve permitir definir se a escala é circular.

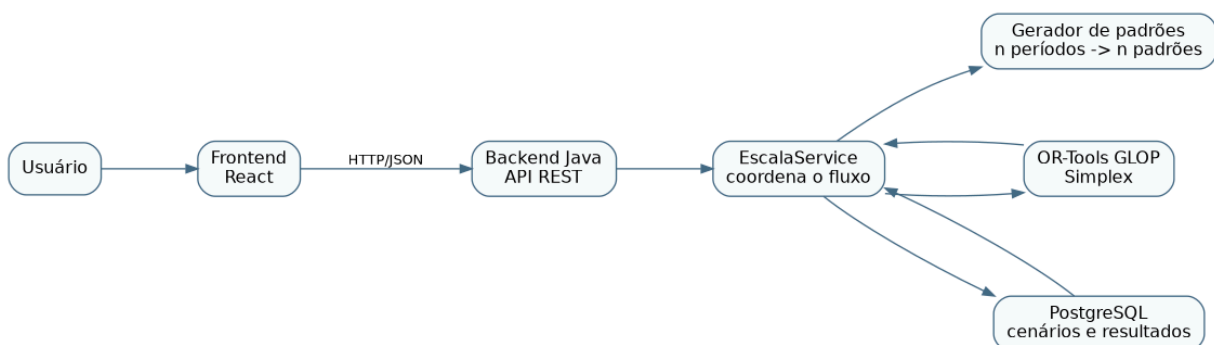
RF07	O sistema deve gerar padrões possíveis de escala a partir das regras informadas.
RF08	O sistema deve permitir pré-visualizar os padrões gerados antes da resolução do problema.
RF09	O sistema deve resolver o problema usando OR-Tools GLOP.
RF10	O sistema deve exibir o valor de Z, representando o total mínimo linear de funcionários.
RF11	O sistema deve exibir a quantidade de funcionários por padrão de escala.
RF12	O sistema deve exibir a cobertura e a sobra por período.
RF13	O sistema deve apresentar gráfico comparando demanda mínima, atendidos e sobra por período.
RF14	O sistema deve exibir o modelo matemático textual gerado.
RF15	O sistema deve informar quando o problema não possuir solução viável.
RF16	O sistema deve validar os dados informados antes de executar o solver.

10. Requisitos não funcionais

Código	Requisito
RNF01	O backend deve ser desenvolvido em Java.
RNF02	O frontend deve ser desenvolvido em React.
RNF03	A comunicação entre frontend e backend deve ocorrer por API REST.
RNF04	O solver utilizado deve ser o Google OR-Tools GLOP.
RNF05	O backend deve isolar a lógica do solver em um serviço próprio.
RNF06	O sistema deve retornar os resultados em formato JSON.
RNF07	O sistema deve validar entradas inválidas antes de tentar resolver o problema.
RNF08	A aplicação deve ser organizada de forma compatível com arquitetura MVC no backend.
RNF09	A interface deve apresentar os resultados de forma visual e compreensível.
RNF10	O sistema deve deixar claro que GLOP resolve programação linear contínua.

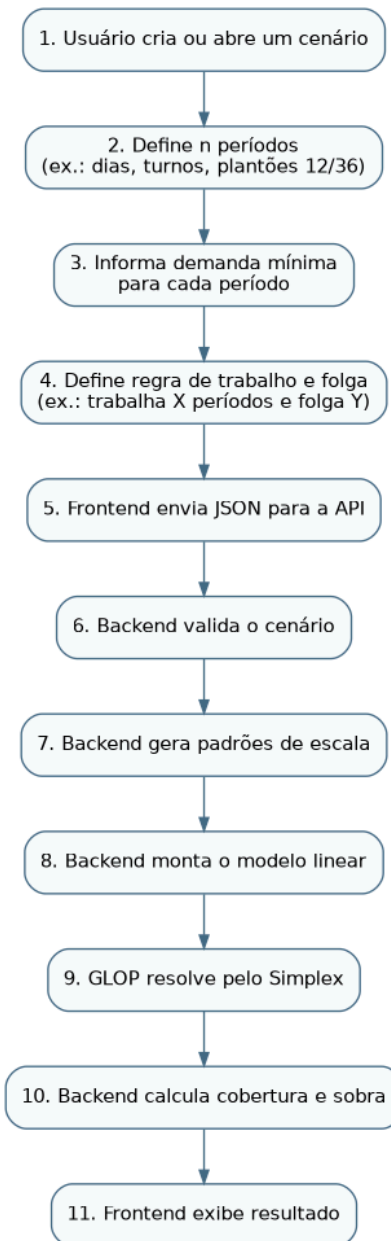
11. Arquitetura geral

A arquitetura separa a interface web, a API Java, os serviços de regra de negócio, a geração de padrões, o solver e a persistência.



12. Fluxo de uso

O fluxo abaixo representa o caminho principal desde a criação do cenário até a exibição dos resultados.



13. Telas mínimas do frontend

1. **Tela inicial:** apresenta os cenários gerados anteriormente e possui botão para criar um novo cenário. Cada item da lista permite abrir o cenário escolhido.
2. **Tela do cenário individual:** usada tanto para criação quanto para edição. Nela o usuário informa nome, descrição, cadastra e ordena os n períodos, informa a demanda mínima de cada período, configura a regra de trabalho/folga, visualiza os padrões gerados e aciona a otimização.
3. **Tela de resultado:** mostra Z, quantidade por padrão, cobertura por período, gráficos, status da solução e modelo matemático textual.

Essa organização reduz o número de telas e deixa o fluxo mais simples: o usuário entra pela lista de cenários, edita ou cria um cenário em uma tela única e depois visualiza o resultado calculado.

14. Contratos de API REST

O frontend não deve montar a matriz do problema nem calcular a solução. Ele envia os dados de entrada para que o backend gere os padrões, monte o modelo e resolva com o solver.

Método	Rota	Função
GET	/api/v1/scenarios	Listar cenários existentes.
POST	/api/v1/scenarios	Criar um cenário.
GET	/api/v1/scenarios/{id}	Buscar um cenário específico.
PUT	/api/v1/scenarios/{id}	Atualizar dados, períodos e regra do cenário.
POST	/api/v1/scenarios/{id}/patterns/preview	Gerar e visualizar padrões antes de resolver.
POST	/api/v1/scenarios/{id}/solve	Resolver o problema de escala.

Dados principais enviados pelo frontend:

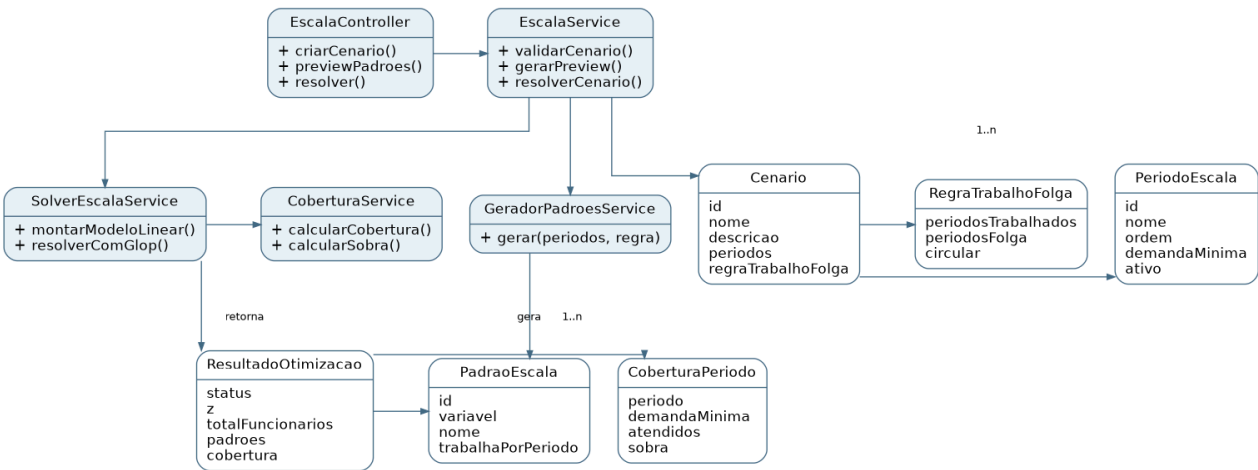
- **name:** nome do cenário.
- **description:** descrição opcional.
- **periods:** lista com n períodos contendo id, nome, ordem, ativo e demanda mínima.
- **workRule:** regra com períodos trabalhados, períodos de folga e indicação de ciclo circular.
- **objective:** objetivo de minimizar o total de funcionários.

Dados principais retornados pelo backend:

- **status:** situação da solução, como OPTIMAL, INFEASIBLE ou ERROR.
- **z:** valor da função objetivo.
- **patterns:** padrões gerados e quantidade de funcionários em cada variável.
- **coverage:** demanda, atendimento e sobra por período.
- **mathematicalModel:** função objetivo, restrições e não negatividade em formato textual.

15. Diagrama de classes

O diagrama abaixo representa as principais classes e serviços do backend, mantendo a separação entre controller, service, geração de padrões, solver, cobertura e objetos de domínio.



16. Responsabilidades principais no backend

- **EscalaController:** expõe os endpoints da API, recebe JSON do frontend, chama o service e retorna resposta em JSON.

- **EscalaService:** coordena o fluxo principal: validar cenário, gerar padrões, chamar solver, calcular cobertura e montar resposta.
- **GeradorPadroesService:** gera padrões possíveis de trabalho e folga a partir dos n períodos.
- **SolverEscalaService:** monta e resolve o modelo matemático com OR-Tools GLOP.
- **CoberturaService:** calcula quantos funcionários trabalham em cada período, compara com a demanda mínima e calcula a sobra.
- **Entidades de domínio:** representam cenário, período de escala, regra, padrão, resultado e cobertura.

17. Validações necessárias

Validações dos períodos:

- A lista de períodos não pode estar vazia.
- Os IDs e ordens dos períodos não podem se repetir.
- A demanda mínima não pode ser negativa.
- Deve existir pelo menos um período ativo.
- Deve existir pelo menos uma demanda maior que zero.
- Os períodos devem ser ordenados pelo campo ordem antes da geração dos padrões.

Validações da regra:

- `periodosTrabalhados` deve ser maior que zero.
- `periodosFolga` deve ser maior que zero.
- A soma entre períodos trabalhados e períodos de folga deve ser menor ou igual ao número de períodos ativos.
- Para escala circular, o sistema deve permitir padrões que cruzam o final da lista de períodos.

18. Mensagens de erro previstas

Código	Quando ocorre
PROBLEMA_INVALIDO	Os dados enviados pelo usuário são inconsistentes.
PERIODOS_INVALIDOS	A lista de períodos está vazia, possui IDs repetidos ou demandas inválidas.
REGRA_INVALIDA	As regras de trabalho e folga são inválidas.
PADROES_NAO_GERADOS	Nenhum padrão válido foi criado.
SOLUCAO_NAO_ENCONTRADA	O solver não encontrou solução viável.
ERRO_SOLVER	Ocorreu uma falha interna ao executar o solver.

19. Visualizações gráficas no frontend

Gráfico de funcionários por padrão: mostra quantos funcionários foram alocados em cada padrão de escala. O tipo recomendado é gráfico de barras.

Padrão	Quantidade
Folga Seq-Ter	0
Folga Ter-Qua	7
Folga Qua-Qui	0
Folga Qui-Sex	4
Folga Sex-Sáb	0
Folga Sáb-Dom	7
Folga Dom-Seg	5

Gráfico de cobertura por período: compara a demanda mínima com a quantidade de funcionários atendendo cada período. O tipo recomendado é gráfico de barras comparativas.

Período	Demanda mínima	Atendidos	Sobra
---------	----------------	-----------	-------

Segunda	18	18	0
Terça	12	16	4
Quarta	15	16	1
Quinta	19	19	0
Sexta	14	19	5
Sábado	16	16	0
Domingo	11	11	0

20. Resultado esperado para o problema original

Variável	Padrão	Quantidade
x1	Folga Segunda e Terça	0
x2	Folga Terça e Quarta	7
x3	Folga Quarta e Quinta	0
x4	Folga Quinta e Sexta	4
x5	Folga Sexta e Sábado	0
x6	Folga Sábado e Domingo	7
x7	Folga Domingo e Segunda	5

Z = 23 funcionários.

Período	Demanda mínima	Atendidos	Sobra
Segunda	18	18	0
Terça	12	16	4
Quarta	15	16	1
Quinta	19	19	0
Sexta	14	19	5
Sábado	16	16	0
Domingo	11	11	0

21. Observação sobre GLOP, Simplex e valores fracionários

O GLOP é um solver de programação linear do OR-Tools. Ele resolve modelos com objetivo linear, restrições lineares e variáveis contínuas. Nesta aplicação, ele será usado para demonstrar o algoritmo Simplex aplicado ao problema de escala.

Isso significa que o resultado pode indicar, por exemplo, $x_1 = 3,5$. Matematicamente, essa solução é válida para o modelo linear contínuo. Porém, em uma escala real de funcionários, não existe meio funcionário. Por isso, o sistema deve apresentar esse ponto de forma didática: resultado ótimo linear, possível arredondamento apenas visual e aviso de que arredondar pode alterar a otimalidade ou a cobertura.

22. Evoluções futuras

- Modo inteiro usando MIP ou CP-SAT.
- Custos diferentes por período ou por padrão.
- Múltiplos tipos de funcionário.
- Restrições específicas para pilotos, motoristas, equipes médicas ou plantões 12/36.
- Cadastro individual de funcionários e disponibilidade.
- Importação de demandas por planilha.
- Exportação para PDF ou Excel.

23. Conclusão

O sistema transforma um problema de escala de funcionários em um modelo de otimização matemática: demanda por período + regras de trabalho e folga + geração de padrões possíveis + solver de otimização = menor quantidade de funcionários necessária.

A mudança para n períodos torna o sistema mais genérico e adequado para semanas comuns, turnos, plantões 12/36 e escalas de pilotos, mantendo a mesma lógica central do modelo.